AIMLT

Mini Project

Submitted to	Date	Sign
Dr. Preeti Kale		

Submitted by	Roll No.	PRN
Nitin Srivastav	45	1032232500
Harsh Patel	58	1032232829
Darshil Patel	41	1032232467
Niharika Sonalkar	62	1032232922

Project Report: RAG-Based Applicant Tracking System

1. Introduction

1.1. Problem Statement

Traditional Applicant Tracking Systems (ATS) often rely heavily on keyword matching to screen resumes. This approach has significant limitations: it struggles to understand the semantic

meaning behind skills and experiences, often overlooks qualified candidates who use different terminology, and lacks the ability for recruiters to interactively query the resume database using natural language. Recruiters may need to manually sift through potentially relevant resumes identified by keyword searches, leading to inefficiency and potential bias.

1.2. Project Goal

The primary goal of this project was to develop an advanced ATS module that leverages Retrieval-Augmented Generation (RAG) to overcome the limitations of traditional systems. The specific objectives were:

- **Enable Natural Language Queries:** Allow hiring managers to ask complex questions about candidates' skills, experience, and qualifications in plain English (e.g., "Which candidates have led a team?", "Find developers with experience in both Python and cloud platforms").
- **Semantic Understanding:** Move beyond keyword matching to understand the underlying meaning and context within resumes, allowing for more accurate identification of relevant skills and experiences even when different phrasing is used.
- **Source Attribution:** Clearly indicate which resume(s) provided the information for any given answer, allowing recruiters to easily verify the context.
- **Interactive Interface:** Provide a user-friendly chat interface for recruiters to interact with the system, view chat history, and access the original source PDFs.

1.3. Relevance

This project directly applies concepts from Natural Language Processing (NLP), Large Language Models (LLMs), vector databases, and modern backend development. It serves as a practical implementation of a RAG pipeline, a crucial technique for building context-aware AI applications, aligning with current industry trends and academic interests in AI engineering.

2. Literature Survey

The development of this RAG-based ATS project draws upon foundational and recent advancements in Natural Language Processing (NLP), Information Retrieval (IR), and Large Language Models (LLMs). The following five papers highlight key concepts and methodologies that underpin the system's design and functionality.

2.1. Dense Passage Retrieval for Open-Domain Question Answering (DPR)

- **Authors:** Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ludwig Schmidt, Edouard Grave, Danqi Chen
- **Published:** EMNLP 2020
- **Relevance:** This paper introduced Dense Passage Retrieval (DPR), a significant advancement in information retrieval. DPR uses dual-encoder architectures to

independently embed questions and passages into dense vector spaces, allowing for efficient and effective retrieval of relevant passages using maximum inner product search (MIPS). Our project directly applies this concept by using `intfloat/e5-base` (a sentence transformer, which is a form of dense encoder) to embed both user queries and resume chunks, enabling semantic similarity search via MongoDB Atlas Vector Search. The move from sparse (keyword) to dense (semantic) retrieval is a core tenet of DPR.

2.2. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (RAG)

- **Authors:** Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Narsimha Chatti, Mike Lewis, Kyunghyun Cho, Brandon Bosworth
- **Published:** NeurIPS 2020
- **Relevance:** This seminal work formally introduced the Retrieval-Augmented Generation (RAG) architecture. RAG addresses the limitations of purely generative LLMs (e.g., hallucination, outdated knowledge) by first retrieving relevant information from a separate corpus and then conditioning a sequence-to-sequence generator on this retrieved context. Our project's entire RAG pipeline, from retrieving top `k` resume chunks to feeding them into Google Gemini, is a direct implementation of the principles laid out in this paper, ensuring answers are grounded in the actual content of the resumes.

2.3. Exploiting Generative Models for Customizable and Explainable Search

- **Authors:** Sewon Min, Mike Lewis, Luke Zettlemoyer, Hannaneh Hajishirzi
- **Published:** ACL 2023
- **Relevance:** This paper explores how generative models can enhance search beyond traditional ranking, particularly focusing on customizability and explainability. It highlights the potential for LLMs to synthesize information from retrieved documents into coherent, summarized answers and to provide direct citations or explanations of *why* certain results are relevant. Our project aligns with this by using Gemini to not just return raw text but to synthesize an answer and, crucially, to cite the `source` filenames directly, enhancing the explainability and verifiability of the system's output.

2.4. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks

- **Authors:** Nils Reimers, Iryna Gurevych
- **Published:** EMNLP 2019
- **Relevance:** Sentence-BERT (SBERT) significantly improved the quality of sentence embeddings, making them more suitable for tasks like semantic similarity search than traditional BERT embeddings. SBERT models generate semantically meaningful sentence embeddings that can be compared directly using cosine similarity, leading to much faster and more accurate retrieval than token-level comparisons. The `intfloat/e5-base` model used in our project is a modern sentence transformer, a

direct descendant of the SBERT architecture, benefiting from these advancements for efficient and effective semantic matching of queries to resume chunks.

2.5. Attention Is All You Need

- **Authors:** Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin
- **Published:** NeurIPS 2017

- **Relevance:** While not directly focused on RAG, this foundational paper introduced the Transformer architecture, which is the backbone of almost all modern LLMs, including BERT (from which SBERT and E5-base are derived) and the Gemini models used for generation. The "attention mechanism" described in the paper allows models to weigh the importance of different words in a sequence, capturing long-range dependencies and complex semantic relationships. Understanding the Transformer architecture provides the underlying theoretical basis for why the embedding models can create semantically rich vectors and why LLMs can generate coherent and context-aware responses.

3. System Architecture

The system employs a multi-stage architecture designed for efficient ingestion, retrieval, and generation.

1. **Data Ingestion:** Resume PDFs are uploaded to a persistent document store (MongoDB Atlas).
2. **Data Processing:** A separate script reads these PDFs, extracts text, splits the text into manageable chunks, generates vector embeddings for each chunk using a sentence transformer model, and stores these chunks along with their vectors and source metadata back into MongoDB Atlas (in a dedicated collection).
3. **Vector Indexing:** MongoDB Atlas Vector Search creates an index on the stored vector embeddings, enabling efficient similarity searches.
4. **Query Handling (RAG Pipeline):**
    - When a user query is received via the API, the query text is converted into a vector embedding using the same sentence transformer model.
    - The vector database (Atlas Vector Search) retrieves the k most semantically similar text chunks based on the query vector.
    - These retrieved chunks (the "context") and the original user query are formatted into a prompt.
    - The prompt is sent to a Large Language Model (LLM - Google Gemini).
    - The LLM generates a natural language answer based *only* on the provided context and the query.

5. **API Layer (FastAPI):** A backend API exposes endpoints for handling user queries and retrieving source PDF documents. It manages the connection to the database and the RAG pipeline components.
6. **Frontend Interface (HTML/CSS/JS):** A web-based chat interface allows users to interact with the API, view responses, see source document links, manage chat history, and view the original PDFs.

4. Backend Implementation

4.1. Data Storage (MongoDB Atlas)

- **Choice Justification:** MongoDB Atlas was selected as the primary data store due to its flexible document model, cloud-native architecture, generous free tier (M0), and crucially, its integrated **Atlas Vector Search** capability. This eliminated the need for a separate vector database and simplified the architecture.
- **Collections:** Two distinct collections within the `ats_db` database were used:
  - **`resumes` Collection:** Stores the original resume documents. Each document contains the `filename` (string) and `file_data` (BSON Binary data representing the raw PDF). This ensures the original source document is always available for retrieval and verification.
  - **`resume_chunks` Collection:** Stores the processed data for searching. Each document represents a single text chunk and contains:
    - `source` (string): The filename of the original PDF this chunk came from (essential metadata for RAG).
    - `text` (string): The actual text content of the chunk.
    - `embedding` (array of numbers): The vector embedding representing the semantic meaning of the `text` field. This is the field indexed by Atlas Vector Search.

4.2. Data Ingestion & Processing (`ingest_data.py`, `ingest_chunks_vectors.py`)

Two separate Python scripts handle the data pipeline:

- **PDF Upload (`ingest_data.py`):** This script reads PDF files from a specified local directory, connects to the MongoDB `resumes` collection, and uploads each PDF as binary data, skipping duplicates based on filename. It utilizes `python-dotenv` to securely load the MongoDB connection string (`MONGO_URI`) from a `.env` file.
- **Chunking & Embedding (`ingest_chunks_vectors.py`):** This script performs the core processing for the RAG system:

- **Connects** to both the `resumes` and `resume_chunks` collections in MongoDB Atlas.
- **Clears** the `resume_chunks` collection to ensure freshness on each run.
- **Retrieves** all documents (PDF binary data) from the `resumes` collection.
- **Parses PDFs:** Uses the `pdfplumber` library (via `io.BytesIO` to read binary data in memory) to extract text content from each PDF page. Includes basic error handling for potentially corrupted or image-based PDFs.
- **Chunks Text:** Employs `langchain.text_splitter.RecursiveCharacterTextSplitter`. After initial experimentation with smaller chunks (`chunk_size=200`), which led to insufficient context for the LLM, the parameters were tuned to `chunk_size=1000` and `chunk_overlap=150`. This ensures that individual chunks contain more meaningful semantic units (like full paragraphs or job descriptions), significantly improving the quality of retrieved context.
- **Generates Embeddings:** Uses `langchain_huggingface.HuggingFaceEmbeddings` with the `intfloat/e5-base` sentence transformer model. This model generates 768-dimensional vectors capturing the semantic meaning of each text chunk. The `embed_documents` method efficiently generates embeddings for all chunks of a single resume at once.
- **Stores Chunks & Vectors:** Inserts the processed data into the `resume_chunks` collection. Each document includes the `text` chunk, its corresponding `embedding` vector, and the `source` filename metadata.

4.3. Vector Database (MongoDB Atlas Vector Search)

- **Index Configuration:** A Vector Search index named `vector_index` was created via the Atlas UI on the `ats_db.resume_chunks` collection. The index definition specifically targets the `embedding` field, configured with `numDimensions: 768` (matching the `e5-base` model) and `similarity: "cosine"` (a standard metric for comparing text embeddings). Debugging involved ensuring the index was active, targeted the correct collection/field, and had the correct dimensions.

4.4. Retrieval-Augmented Generation (RAG) Pipeline (`main.py` using `langchain`)

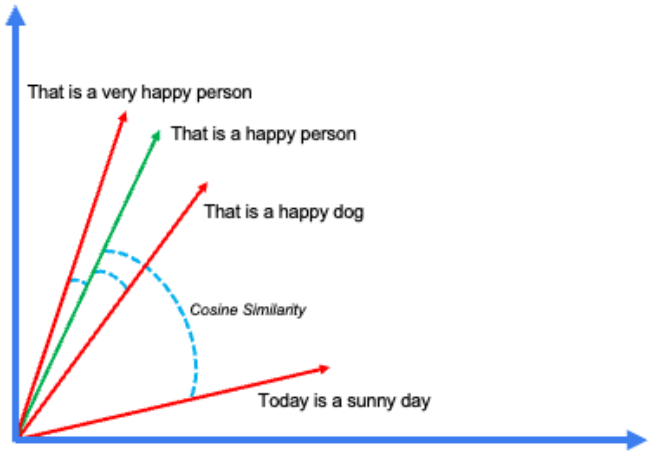
The core RAG logic is orchestrated using LangChain within the FastAPI application's `lifespan` function for efficiency.

- **Retriever (`langchain_mongodb.MongoDBAtlasVectorSearch.as_retriever`):** An instance of `MongoDBAtlasVectorSearch` connects to the configured index. The

`.as_retriever()` method is used to create a LangChain Retriever object. This object handles embedding the incoming user query (using the same `HuggingFaceEmbeddings` model) and performing the similarity search against the Atlas Vector Search index. It was configured with `search_kwargs={"k": 10}` to retrieve the top 10 most relevant chunks, providing richer context to the LLM compared to the initial `k=5`.

- **LLM (`langchain_google_genai.ChatGoogleGenerAI`):** The Google Gemini family of models was used for generations. Initial attempts with `gemini-1.5-flash` and `gemini-2.5-pro` encountered API availability issues and restrictive free-tier rate limits (`429 ResourceExhausted`), leading to server startup failures. The final implementation uses the stable `gemini-1.0-pro` model, which proved reliable and had adequate performance for this task.
- **Prompt Engineering (`langchain_core.prompts.PromptTemplate`):** A carefully crafted prompt template guides the LLM's behavior. Key instructions include:
    - Assuming the persona of an expert Hiring Manager.
    - Using *only* the provided `Context` (retrieved chunks) to answer the `Question`.
    - Explicitly citing the `source` filename (available in the context metadata) for each piece of information.
    - Strictly prohibiting fabrication ("DO NOT LIE") and instructing the LLM to state when information cannot be found.

Chain Construction (LangChain Expression Language - LCEL): The components are linked using LCEL's pipe (`|`) syntax. The chain takes the user's question, passes it to the retriever to fetch context, formats the context and question into the prompt, sends the prompt to the LLM, and parses the LLM's output string. This declarative syntax simplifies the definition and execution of the RAG pipeline.



4.5. API Layer (`FastAPI` in `main.py`)

- **Framework:** FastAPI was chosen for its high performance, asynchronous capabilities (used via `async def` and `await`), automatic OpenAPI (Swagger UI) documentation, and Pydantic-based data validation.
- **Lifespan Management:** An `@asynccontextmanager` named `lifespan` is used. Code before the `yield` runs on application startup (loading models, connecting to DB, building the RAG chain), storing these expensive objects in an `app_state` dictionary. Code after the `yield` runs on shutdown (closing the DB connection). This significantly improves performance by avoiding re-initialization on every request.
- **Endpoints:**
  - `POST /query`: Accepts a JSON payload `{ "question": "..." }`. It asynchronously invokes the retriever to get relevant documents (primarily to extract `source` filenames), then invokes the full RAG chain to get the final text answer. It returns a JSON response `{ "answer_text": "...", "source_files": [...] }`. Error handling returns a 500 status with details.
  - `GET /get_pdf/{filename}`: Accepts a filename as a path parameter. It queries the `resumes` collection for the document matching the filename, retrieves the `file_data` (binary PDF), and returns it using `StreamingResponse` with the correct `media_type="application/pdf"`, allowing the browser to display the PDF directly. Includes 404 handling if the file isn't found.
  - `GET /`: Serves the main `static/index.html` file using `FileResponse`.

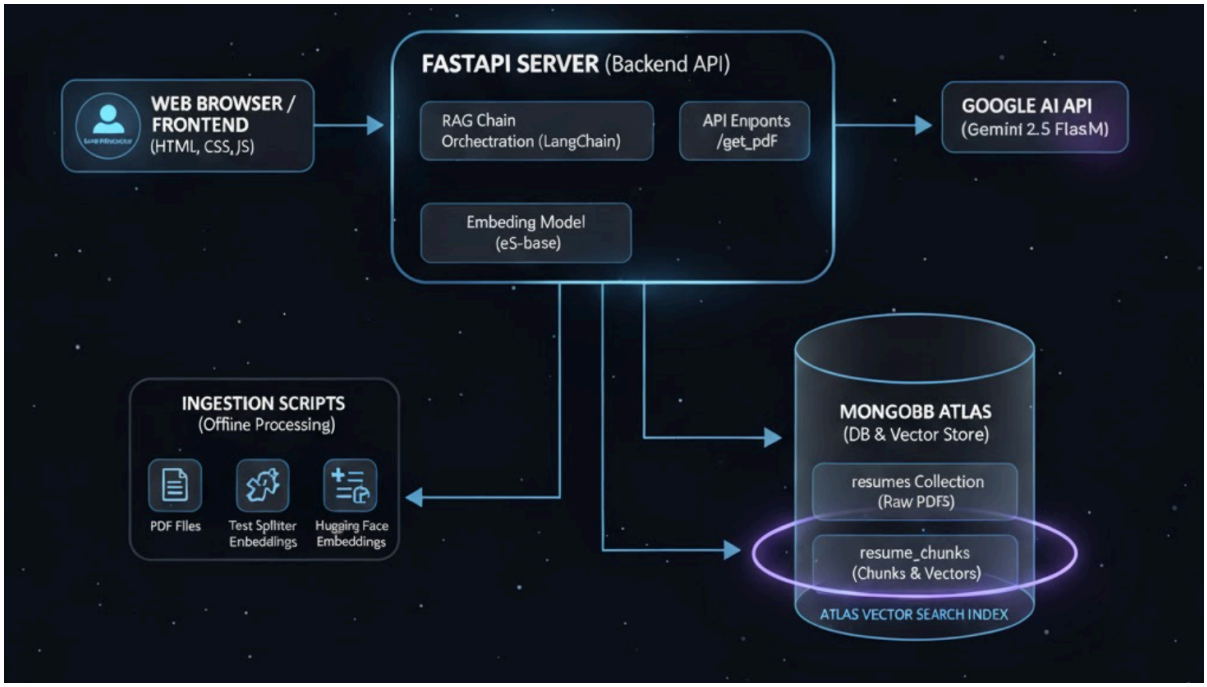
    - `app.mount("/", StaticFiles(directory="static"), name="static")`: Configures FastAPI to serve all files (CSS, JS) from the `static` directory.
- **Secrets Management:** Uses `google.colab.userdata` (in Colab) or `python-dotenv` (in VS Code) to securely load `MONGO_URI` and `GOOGLE_API_KEY` from secrets/`.env` file, keeping credentials out of the source code.

5. Frontend Implementation (`static/` folder)

A single-page web application provides the user interface.

- **Structure (`index.html`):** Defines the main layout using semantic HTML. Key elements include:
    - A `<canvas>` element (`#starfield`) for the animated background.
    - A retractable sidebar (`#sidebar`) containing a "New Chat" button, search input, and chat history list (`#chat-list`).
    - A main content area (`#main-content`) containing toggle buttons for the sidebar, a chat header (`#chat-header`), the message display area (`#chat-messages`), and the user input form (`#chat-form`).
- **Styling (`style.css`):** Implements the visual design:
    - **Dark "Grok-like" Theme:** Uses a black background, dark grey/blue interface elements, and light text.
    - **Starfield Background:** Positions the canvas behind all other content.
    - **Sidebar Styling:** Styles the retractable sidebar, search bar, and chat list items, including hover and active states, and darkens the color scheme.
    - **Floating Glass Chat Bubbles:** Styles the `.message` elements with transparency (`rgba`), borders, `backdrop-filter: blur()`, and soft shadows to create a glassy effect. Removes the containing box (`#chat-container`) so bubbles float directly on the background.
    - **Subtle Animation:** Applies a slow `float` CSS animation to the message bubbles, giving them a gentle, organic movement. Includes variations using `:nth-child` to desynchronize the movement.
- **Logic (`script.js`):** Handles interactivity and communication with the backend:
    - **Starfield Animation:** Implements a JavaScript function (`animateStars`) using `requestAnimationFrame` to draw and update the position/appearance of stars on the canvas, creating a "moving through space" effect.
    - **API Interaction (`fetch`):** Uses the browser's `fetch` API to send POST requests (with the user's question as JSON) to the `/query` endpoint and handle the JSON response. Includes error handling for network or server issues.

- **Chat Display:** Dynamically creates HTML elements for user and bot messages and appends them to the `#chat-messages` container. Scrolls the container to the bottom automatically. Displays a "Thinking..." indicator while waiting for the API response. Parses the `answer_text` and creates clickable links for each filename in `source_files`, pointing to the `/get_pdf/{filename}` endpoint (opening in a new tab).
- **Chat History (`localStorage`):** Stores the entire chat history (an array of chat objects, each with an ID, name, and messages array) in the browser's `localStorage`. Loads history on page start. Creates a new chat session when "New Chat" is clicked, assigning a timestamp as the ID.Saves the current chat state to `localStorage` whenever a message is added.
- **Sidebar Functionality:** Implements event listeners for toggling the sidebar visibility, starting new chats, filtering the chat list based on the search input, and deleting individual chat sessions (with a confirmation prompt). Deleting the active chat loads the next available one or starts a new chat.



6. Development Environment & Deployment

- **Prototyping (Google Colab):** The project was initially developed and debugged in Google Colab. This environment was suitable for rapid iteration on the Python backend logic. Key challenges included managing Python package dependencies (requiring careful installation order and version upgrades), running the FastAPI server reliably (eventually using a background thread approach after issues with `asyncio` loops and

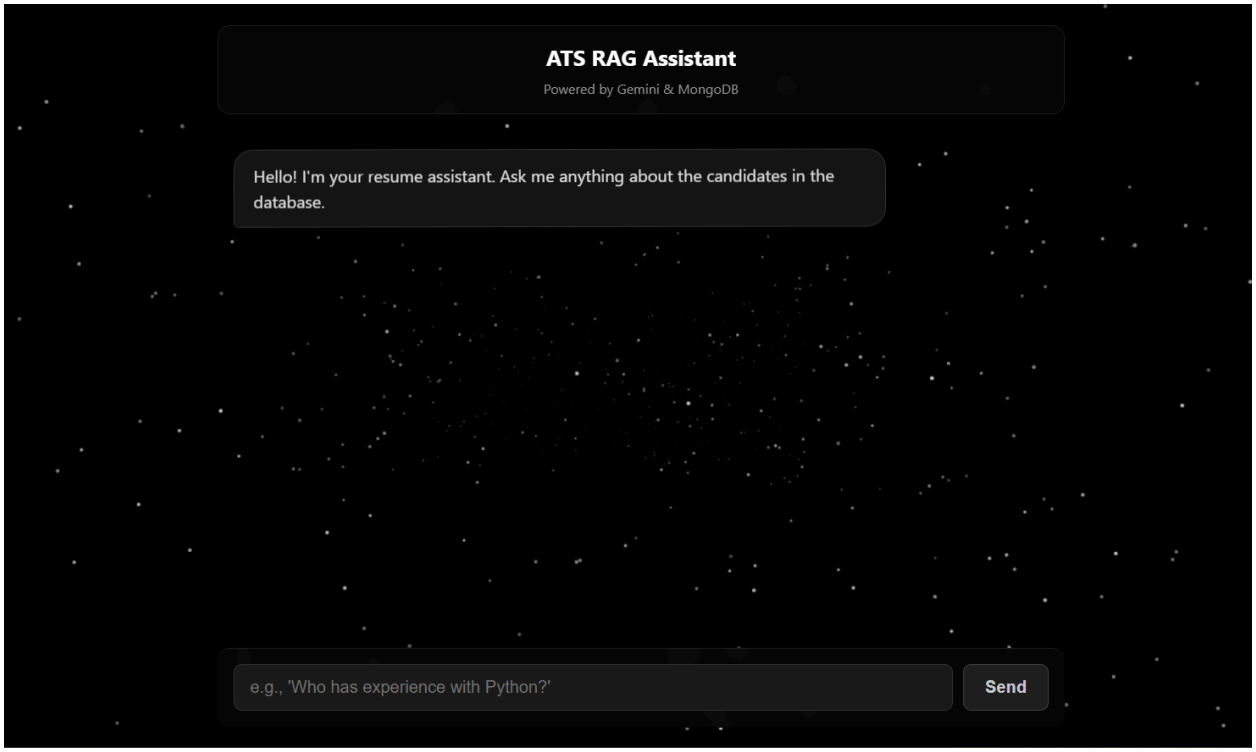
shell commands), and dealing with the ephemeral filesystem (leading to the switch from FAISS to Atlas Vector Search).

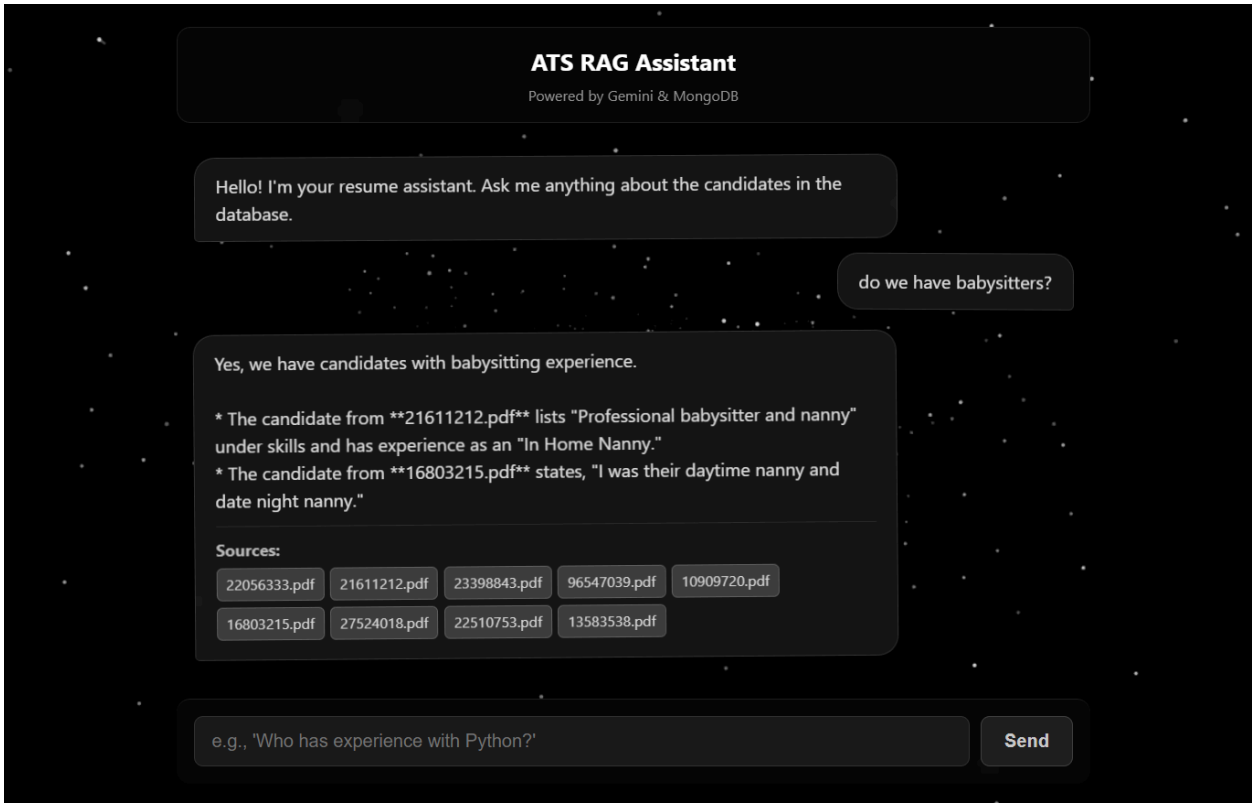
- **Local Development (VS Code):** As the project grew to include frontend files, development transitioned to VS Code. This provided a more robust environment for managing the `main.py`, `.env`, `static/` folder structure and offered better tools for HTML/CSS/JS development.
- **Running the Application:**
    - **Locally:** The application is run using `uvicorn main:app --reload --port 8000` from the VS Code terminal. Access is via `http://localhost:8000`.
    - **During Colab Development:** The `pyngrok` library was used to create a temporary public HTTPS tunnel to the FastAPI server running within the Colab environment, allowing testing from a local browser via an `ngrok.io` URL.

7. Challenges & Future Work

7.1. Challenges Encountered

- **Dependency Management:** Significant time was spent resolving conflicting version requirements between `langchain` components and Google's AI libraries within the Colab environment, requiring careful uninstallation and re-installation sequences.
- **Colab Server Execution:** Running a persistent web server like FastAPI within Colab's notebook environment proved tricky, encountering issues with `asyncio` event loops and background processes being killed prematurely. The final solution involved running Uvicorn in a separate Python thread.
- **API Rate Limits:** The free tier of the Google Gemini API has strict rate limits (e.g., 2 RPM for `gemini-2.5-pro`), which caused server startup failures. Switching to the `gemini-1.0-pro` model resolved this.
- **Retriever Tuning:** Initial RAG results were poor due to small text chunks (`chunk_size=200`) providing insufficient context. Increasing the chunk size (`1000`) and the number of retrieved documents (`k=10`) dramatically improved answer quality.
- **Vector Index Configuration:** Debugging the connection between the retriever and Atlas Vector Search required careful verification of the index status, target collection (`resume_chunks` vs. `resumes`), indexed field path (`embedding`), and dimensionality (`768`).


ATS RAG Assistant
Powered by Gemini & MongoDB
Hello! I'm your resume assistant. Ask me anything about the candidates in the database.
e.g., 'Who has experience with Python?'
Send


ATS RAG Assistant
Powered by Gemini & MongoDB
Hello! I'm your resume assistant. Ask me anything about the candidates in the database.
do we have babysitters?
Yes, we have candidates with babysitting experience.
* The candidate from **21611212.pdf** lists "Professional babysitter and nanny" under skills and has experience as an "In Home Nanny."
* The candidate from **16803215.pdf** states, "I was their daytime nanny and date night nanny."
Sources:
22056333.pdf
21611212.pdf
23398843.pdf
96547039.pdf
10909720.pdf
16803215.pdf
27524018.pdf
22510753.pdf
13583538.pdf
e.g., 'Who has experience with Python?'
Send

7.2. Potential Future Work

- **Advanced Skill Extraction:** Implement a dedicated Named Entity Recognition (NER) model (potentially fine-tuned on resume data) to explicitly extract skills, experience years, company names, etc., and store them as structured metadata alongside the chunks in MongoDB. This would allow for more precise filtering combined with semantic search (e.g., "Find candidates with 5+ years of Python experience").
- **Evaluation Framework:** Develop metrics to evaluate the performance of both the retrieval step (e.g., Hit Rate, MRR) and the generation step (e.g., faithfulness, relevance).
- **Job Description Matching:** Integrate functionality to ingest job descriptions, generate embeddings for them, and calculate similarity scores against candidate resume chunks/embeddings to rank candidates for specific roles.
- **Cloud Deployment:** Deploy the FastAPI application and potentially the ingestion scripts to a scalable cloud platform (e.g., Google Cloud Run, AWS Lambda + API Gateway, Heroku) for persistent availability.
- **User Authentication:** Add user accounts and authentication to manage access if the tool were to be used by multiple recruiters.

8. Conclusion

This project successfully demonstrated the development of an enhanced Applicant Tracking System module utilizing a Retrieval-Augmented Generation (RAG) architecture. By integrating MongoDB Atlas Vector Search, LangChain, Google Gemini, and FastAPI, the system enables recruiters to interact with a large corpus of resumes using natural language queries, receiving contextually relevant and source-grounded answers. The transition from local file-based vector storage (FAISS) to a cloud-native solution (Atlas Vector Search) addressed persistence issues and provided a scalable foundation. Despite challenges related to dependency management and API limitations, the final application provides a significantly improved user experience over traditional keyword-based ATS methods, showcasing the power of combining semantic search with generative AI for information retrieval tasks in the HR domain.